Programming Languages Semantics

Ethan Zhang, Srujan Roplekar

CS SAIL 2026

What are Semantics?

There are two parts to every programming language: the *syntax* and the *semantics*.

Syntax describes the *structure* of code (i.e. what is an allowed sequence of symbols). For example, the sequence `5 + 3` is allowed, but `5 + (` is not.

Semantics assigns *meaning* to code. For example, we might define the meaning of `x = 5 + 3` to be "compute the result of `5 + 3` and store it in the variable `x`."

Note that there may be sequences of symbols that are structurally correct but meaningless, e.g. `5 = 5 + "hello, world"`.

Why study Semantics?

- Who implements our programming languages?
  - Understand what code 'means' *before* writing compiler/intepreter!
- How do we check our programs?
  - 'Program verification'

Why study Semantics?

- Who implements our programming languages?
  - Understand what code 'means' *before* writing compiler/intepreter!
- How do we check our programs?
  - 'Program verification'

Why study Semantics?

```
>>> x = 3
>>> if (x > 0):
...     print('apple')
... else:
...     print('banana')
...
cucumber
```

Why study Semantics?

```
>>> x = 3
>>> if (x > 0):
...     print('apple')
... else:
...     print('banana')
...
cucumber
```

Why study Semantics?

- Who implements our programming languages?
  - Understand what code 'means' *before* writing compiler/intepreter!
- How do we check our programs?
  - 'Program verification'

Why study Semantics?

- Who implements our programming languages?
  - Understand what code 'means' *before* writing compiler/intepreter!
- How do we check our programs?
  - 'Program verification'

Why study Semantics?

- Who implements our programming languages?
  - Understand what code 'means' *before* writing compiler/intepreter!
- How do we check our programs?
  - 'Program verification'

Why study Semantics?

Figure 1: Self-destruction of Ariane 5, flight V88

Why study Semantics?

The launcher started to disintegrate at about H0 + 39 seconds...the software exception was caused during...conversion from 64-bit floating point to 16-bit signed integer.

– Ariane 5 Flight 501 Failure, Report by the Inquiry Board

$337 million floating point error!

Why study Semantics?

The launcher started to disintegrate at about H0 + 39 seconds...the software exception was caused during...conversion from 64-bit floating point to 16-bit signed integer.

– Ariane 5 Flight 501 Failure, Report by the Inquiry Board

$337 million floating point error!

Overview

Arithmetic Expressions: Syntax

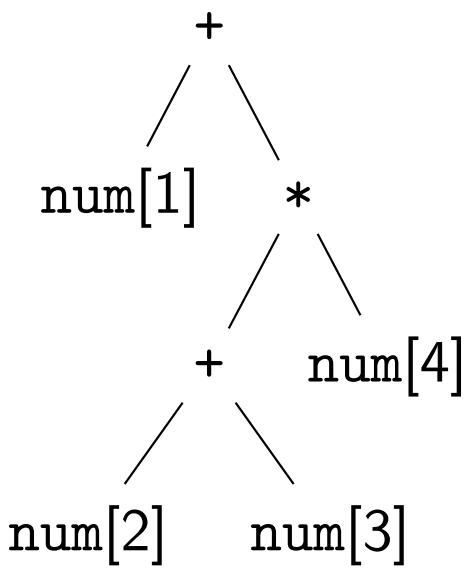
$$
\begin{array}{rcll}
e & ::= & \texttt{num}[n] & \textit{Numeric constants} \\
 & | & e + e & \textit{Addition} \\
 & | & e * e & \textit{Multiplication}
\end{array}
$$

In English, an expression e is:

- a numeric constant $\texttt{num}[n]$ where n is a number,
- a sum $e + e$ where each e is an expression, or
- a product $e * e$ where each e is an expression.

Arithmetic Expressions: Example

`1 + ((2 + 3) * 4)` $\longmapsto$



Aside: (Big-step) Operational Semantics

The goal of *operational semantics* is to describe a program by **what it does**.

But what does this mean? One idea is to think of programs as *functions*: given some inputs, we get some output.

Following this intuition, big-step semantics describes a program by its final result. For example, the program `1 + ((2 + 3) * 4)` is described by the number 21.

Aside: Inference Rules

As computer scientists, we want to describe a program's semantics by formal rules so that we can reason about them mathematically and implement them in code.

Following the tradition of mathematical logic, semantics are described by *inference rules*:

$$\frac{J_1 \quad J_2 \quad \dots \quad J_n}{J} \ (\textsc{Rule-Name})$$

In English, we say that "J holds, provided that $J_1, J_2, \dots, J_n$ all hold."

Arithmetic Expressions: Semantics

$$\frac{}{\mathbf{num}[n] \Downarrow \mathbf{num}[n]}\ (\textsc{Const})$$

$$\frac{e_1 \Downarrow \mathbf{num}[n] \quad e_2 \Downarrow \mathbf{num}[m]}{e_1 + e_2 \Downarrow \mathbf{num}[n+m]}\ (\textsc{Add}) \qquad \frac{e_1 \Downarrow \mathbf{num}[n] \quad e_2 \Downarrow \mathbf{num}[m]}{e_1 * e_2 \Downarrow \mathbf{num}[n \cdot m]}\ (\textsc{Mul})$$

Arithmetic Expressions: Semantics

$$\frac{}{\mathtt{num}[n] \Downarrow \mathtt{num}[n]}\ (\textsc{Const})$$

$$\frac{e_1 \Downarrow \mathtt{num}[n] \quad e_2 \Downarrow \mathtt{num}[m]}{e_1 + e_2 \Downarrow \mathtt{num}[n+m]}\ (\textsc{Add})$$

$$\frac{e_1 \Downarrow \mathtt{num}[n] \quad e_2 \Downarrow \mathtt{num}[m]}{e_1 * e_2 \Downarrow \mathtt{num}[n \cdot m]}\ (\textsc{Mul})$$

Arithmetic Expressions: Semantics

$$\frac{}{\mathtt{num}[n] \Downarrow \mathtt{num}[n]}\ (\textsc{Const})$$

$$\frac{e_1 \Downarrow \mathtt{num}[n] \quad e_2 \Downarrow \mathtt{num}[m]}{e_1 + e_2 \Downarrow \mathtt{num}[n+m]}\ (\textsc{Add})$$

$$\frac{e_1 \Downarrow \mathtt{num}[n] \quad e_2 \Downarrow \mathtt{num}[m]}{e_1 * e_2 \Downarrow \mathtt{num}[n \cdot m]}\ (\textsc{Mul})$$

Arithmetic Expressions: Semantics

$$\frac{}{\texttt{num}[n] \Downarrow \texttt{num}[n]}\ (\textsc{Const})$$

$$\frac{e_1 \Downarrow \texttt{num}[n] \quad e_2 \Downarrow \texttt{num}[m]}{e_1 + e_2 \Downarrow \texttt{num}[n+m]}\ (\textsc{Add}) \qquad \frac{e_1 \Downarrow \texttt{num}[n] \quad e_2 \Downarrow \texttt{num}[m]}{e_1 * e_2 \Downarrow \texttt{num}[n \cdot m]}\ (\textsc{Mul})$$

Adding Variables: Syntax

$$
\begin{array}{rcll}
e & ::= & \texttt{num}[n] & \textit{Numeric constants} \\
 & | & x & \textit{Variable reference} \\
 & | & e + e & \textit{Addition} \\
 & | & e * e & \textit{Multiplication} \\
 & | & \texttt{let } x \texttt{ = } e \texttt{ in } e & \textit{Variable definition}
\end{array}
$$

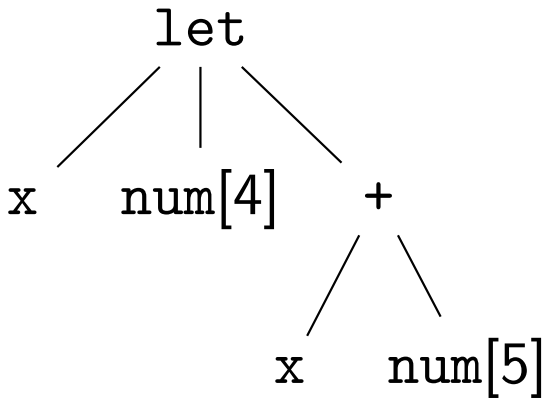
In English, an expression e is:

- a numeric constant $\texttt{num}[n]$ where n is a number,
- a variable reference x where x is an identifier,
- a sum e + e where each e is an expression,
- a product e * e where each e is an expression, or
- a "let" expression, where we define a variable x.

Adding Variables: Example

```
let x = 4
 in (x + 5)
```

$\longmapsto$



Adding Variables: Context

Now that we have variables, the output of a program depends not only on the source code but also on the *environment* in which it is executed.

For example, the output of the program `x + 5` depends on the current value of x!

Hence, we introduce an *execution context* to keep track of all the inputs we need.

Adding Variables: Context

Previously, the context only contained the program's source code. Now, it also contains the values assigned to each variable name:

$$\langle \text{program}, \text{environment} \rangle$$

Adding Variables: Semantics

$$\frac{}{\langle \mathtt{num}[n], S\rangle \Downarrow \mathtt{num}[n]}\ (\textsc{Const}) \qquad \frac{x = \mathtt{num}[n] \text{ in } S}{\langle x, S\rangle \Downarrow \mathtt{num}[n]}\ (\textsc{Var})$$

$$\frac{\langle e_1, S\rangle \Downarrow \mathtt{num}[m] \quad \langle e_2, S\rangle \Downarrow \mathtt{num}[n]}{\langle e_1 + e_2, S\rangle \Downarrow \mathtt{num}[m+n]}\ (\textsc{Add})$$

$$\frac{\langle e_1, S\rangle \Downarrow \mathtt{num}[m] \quad \langle e_2, S\rangle \Downarrow \mathtt{num}[n]}{\langle e_1 * e_2, S\rangle \Downarrow \mathtt{num}[m \cdot n]}\ (\textsc{Mul})$$

$$\frac{\langle e_1, S\rangle \Downarrow \mathtt{num}[m] \quad \langle e_2, S \cup \{x = \mathtt{num}[m]\}\rangle \Downarrow \mathtt{num}[n]}{\langle \mathtt{let}\ x\ \mathtt{=}\ e_1\ \mathtt{in}\ e_2, S\rangle \Downarrow \mathtt{num}[n]}\ (\textsc{Let})$$

Adding Variables: Semantics

$$\frac{}{\langle \texttt{num}[n], S\rangle \Downarrow \texttt{num}[n]}\ (\textsc{Const}) \qquad \frac{x = \texttt{num}[n] \text{ in } S}{\langle x, S\rangle \Downarrow \texttt{num}[n]}\ (\textsc{Var})$$

$$\frac{\langle e_1, S\rangle \Downarrow \texttt{num}[m] \quad \langle e_2, S\rangle \Downarrow \texttt{num}[n]}{\langle e_1 + e_2, S\rangle \Downarrow \texttt{num}[m+n]}\ (\textsc{Add})$$

$$\frac{\langle e_1, S\rangle \Downarrow \texttt{num}[m] \quad \langle e_2, S\rangle \Downarrow \texttt{num}[n]}{\langle e_1 * e_2, S\rangle \Downarrow \texttt{num}[m \cdot n]}\ (\textsc{Mul})$$

$$\frac{\langle e_1, S\rangle \Downarrow \texttt{num}[m] \quad \langle e_2, S \cup \{x = \texttt{num}[m]\}\rangle \Downarrow \texttt{num}[n]}{\langle \texttt{let } x \texttt{ = } e_1 \texttt{ in } e_2, S\rangle \Downarrow \texttt{num}[n]}\ (\textsc{Let})$$

NEXT LOCATION

CLASS #2
CHECK YOUR SCHEDULE!

(END OF 10AM CLASSES)